

Secure Software Development

Penetrate and Patch

- ❑ Usual approach to software development
 - Develop product as quickly as possible
 - Release it without adequate testing
 - Patch the code as flaws are discovered
- ❑ In security, this is “penetrate and patch”
 - A **bad** approach to software development
 - An **even worse** approach to secure software!

Why Penetrate and Patch?

- ❑ First to market advantage
 - First to market likely to become market leader
 - Market leader has huge advantage in software
 - Users find it safer to “follow the leader”
 - Boss won't complain if your system has a flaw, as long as everybody else has same flaw...
- ❑ Sometimes called “network economics”

Why Penetrate and Patch?

- ❑ Secure software development is hard
 - Costly and time consuming development
 - Costly and time consuming testing
 - Cheaper to let customers do the work!
- ❑ No serious economic disincentive
 - Even if software flaw causes major losses, the software vendor is not liable

Penetrate and Patch Fallacy

- ❑ **Fallacy:** If you keep patching software, eventually it will be secure
- ❑ Why is this a fallacy?
 - Empirical evidence to the contrary
 - Patches often add new flaws
 - Software is a moving target: new versions, features, changing environment, new uses,...

Open vs Closed Source

- ❑ Open source software
 - The source code is available to user
 - For example, Linux
- ❑ Closed source
 - The source code is not available to user
 - For example, Windows

Open Source Security

- ❑ Claimed advantages of open source is
 - **More users:** more people looking at the code should imply fewer flaws
 - A variant on Kerchoffs Principle
- ❑ Is this valid?
 - How many “**users**” looking for security flaws?
 - How many “**users**” focused on boring parts?
 - How many “**users**” belong to security experts?
 - Attackers can also look for flaws!
 - Evil coder might be able to insert a flaw

Open Source Security

- ❑ Open source example: wu-ftp
 - About 8,000 lines of code
 - A security-critical application
 - Was deployed and widely used
 - After 10 years, serious security flaws discovered!
- ❑ More generally, open source software has done little to reduce security flaws
- ❑ Why?
 - Open source follows penetrate and patch model!

Closed Source Security

- ❑ Claimed advantage of closed source
 - Security flaws not as visible to attacker
 - This is a form of "security by obscurity"
- ❑ Is this valid?
 - Many exploits do not require source code
 - Possible to analyze closed source code...
 - ...though it is a lot of work!
 - Is "security by obscurity" real security?

Open vs Closed Source

- ❑ No obvious security advantage to either open or closed source
- ❑ More significant than open vs closed source is software development practices
- ❑ Both open and closed source follow the "penetrate and patch" model

Security and Testing

- ❑ The fundamental problem
 - Good guys must find (almost) all flaws
 - Bad guy only needs 1 (exploitable) flaw
- ❑ Software reliability far more difficult in security than elsewhere

Security Testing: Do the Math

- ❑ Expect to find 1 bug for every 10^3 hours testing
- ❑ Good guys spend 10^7 hours testing: **find 10^4 bugs**
 - Good guys have found 1% of all the bugs
- ❑ Trudy spends 10^3 hours of testing: **finds 1 bug**
- ❑ Chance good guys found Trudy's bug is only **1% !!!**

Software Development

- General software development model
 - Specify
 - Design
 - Implement
 - Test
 - Review
 - Document
 - Manage
 - Maintain



Secure Software Development

- ❑ Goal: move away from “penetrate and patch”
- ❑ Penetrate and patch will always exist
 - But if more care taken in development, then fewer and less severe flaws to patch
- ❑ Secure software development not easy
- ❑ Much more time and effort required thru entire development process
- ❑ Today, little economic incentive for this!

Secure Software Development

- We briefly discuss the following
 - Design
 - Hazard analysis
 - Peer review
 - Testing
 - Configuration management
 - Postmortem for mistakes

Design

- ❑ Careful initial design
- ❑ Try to avoid high-level errors
 - Such errors may be impossible to correct later
 - Certainly costly to correct these errors later
- ❑ Verify assumptions, protocols, etc.
- ❑ Usually informal approach is used
- ❑ Formal methods
 - Possible to rigorously **prove** design is correct
 - In practice, only works in simple cases

Hazard Analysis

- ❑ Hazard analysis (or threat modeling)
 - Develop hazard list
 - List of what ifs
 - Schneier's "attack tree"
 - Attack trees are conceptual diagrams showing how an asset, or target, might be attacked
- ❑ Many formal approaches
 - Hazard and operability studies (HAZOP)
 - is a structured and systematic examination of a planned or existing process or operation in order to identify and evaluate problems that may represent risks to personnel or equipment, or prevent efficient operation.
 - Failure modes and effective analysis (FMEA)
 - a structured way to identify and address potential problems, or failures and their resulting effects on the system or process before an adverse event occurs
 - Fault tree analysis (FTA) or event tree analysis
 - a method of identifying the possible causes of a system failure. A fault tree is used to graphically illustrate the different potential causes of a failure in the form of a diagram.

Peer Review

- ❑ Three levels of peer review
 - Review (informal)
 - Walk-through (semi-formal)
 - Inspection (formal)
- ❑ Each level of review is important
- ❑ Much evidence that peer review is effective
- ❑ Although programmers might not like it!

Levels of Testing

- ❑ Module testing — test each small section of code
- ❑ Component testing — test combinations of a few modules
- ❑ Unit testing — combine several components for testing
- ❑ Integration testing — put everything together and test

Types of Testing

- ❑ Function testing — verify that system functions as it is supposed to
- ❑ Performance testing — other requirements such as speed, resource use, etc.
- ❑ Acceptance testing — customer involved
- ❑ Installation testing — test at install time
- ❑ Regression testing — test after any change

Other Testing Issues

- ❑ Active fault detection
 - Don't wait for system to fail
 - Actively try to make it fail — attackers will!
- ❑ Fault injection
 - Insert faults into the process
 - Even if no obvious way for such a fault to occur
- ❑ Bug injection
 - Insert bugs into code
 - See how many of injected bugs are found
 - Can use this to estimate number of bugs
 - Assumes injected bugs similar to unknown bugs

Security Testing: The Bottom Line

- ❑ **Security testing** is far more demanding than non-security testing
- ❑ Non-security testing — does system do what it is supposed to?
- ❑ Security testing — does system do what it is supposed to **and nothing more?**
- ❑ Usually impossible to do exhaustive testing

Security Testing: The Bottom Line

- ❑ How much testing is enough?
- ❑ Seems to imply testing is nearly hopeless!
- ❑ But there is some hope...
 - If we eliminate an entire class of flaws then statistical model breaks down
 - For example, if a single test (or a few tests) find all buffer overflows

Configuration Issues

- ❑ Types of changes
 - Minor changes — maintain daily functioning
 - Adaptive changes — modifications
 - Perfective changes — improvements
 - Preventive changes — no loss of performance
- ❑ Any change can introduce new flaws!

Postmortem

- ❑ After fixing any security flaw...
- ❑ Carefully analyze the flaw
- ❑ To learn from a mistake
 - Mistake must be analyzed and understood
 - Must make effort to avoid repeating mistake
- ❑ In security, **always** learn more when things go wrong than when they go right
- ❑ Postmortem may be the most under-used tool in all of security engineering!

Software and Security

- ❑ Even with best development practices, security flaws will still exist
- ❑ Absolute security is (almost) never possible
- ❑ So, it is not surprising that absolute software security is impossible
- ❑ The goal is to minimize and manage risks of software flaws
- ❑ Do not expect dramatic improvements in consumer software security anytime soon!

Operating Systems and Security

OS and Security

- ❑ OSs are large, complex programs
 - Many bugs in any such program
 - We have seen that bugs can be security threats
- ❑ Here we are concerned with security provided by OS
 - Not concerned with threat of bad OS software
- ❑ Concerned with OS as security **enforcer**

OS Security Challenges

- ❑ Modern OS is **multi-user** and **multi-tasking**
- ❑ OS must deal with
 - Memory
 - I/O devices (disk, printer, etc.)
 - Programs, threads
 - Network issues
 - Data, etc.
- ❑ OS must protect processes from other processes and users from other users
 - Whether accidental or malicious

OS Security Functions

- ❑ Memory protection
 - Protect memory from users/processes
- ❑ File protection
 - Protect user and system resources
- ❑ Authentication
 - Determines and enforce authentication results
- ❑ Authorization
 - Determine and enforces access control

Memory Protection

- ❑ Fundamental problem
 - How to keep users/processes separate?
- ❑ Separation
 - Physical separation — separate devices
 - Temporal separation — one at a time
 - Logical separation — sandboxing, etc.
 - Cryptographic separation — make information unintelligible to outsider
 - Or any combination of the above

Memory Protection

- ❑ **Fence** — users cannot cross a specified address
 - Static fence — fixed size OS
 - Dynamic fence — fence register
- ❑ Base/bounds register — lower and upper address limit
- ❑ Assumes contiguous space



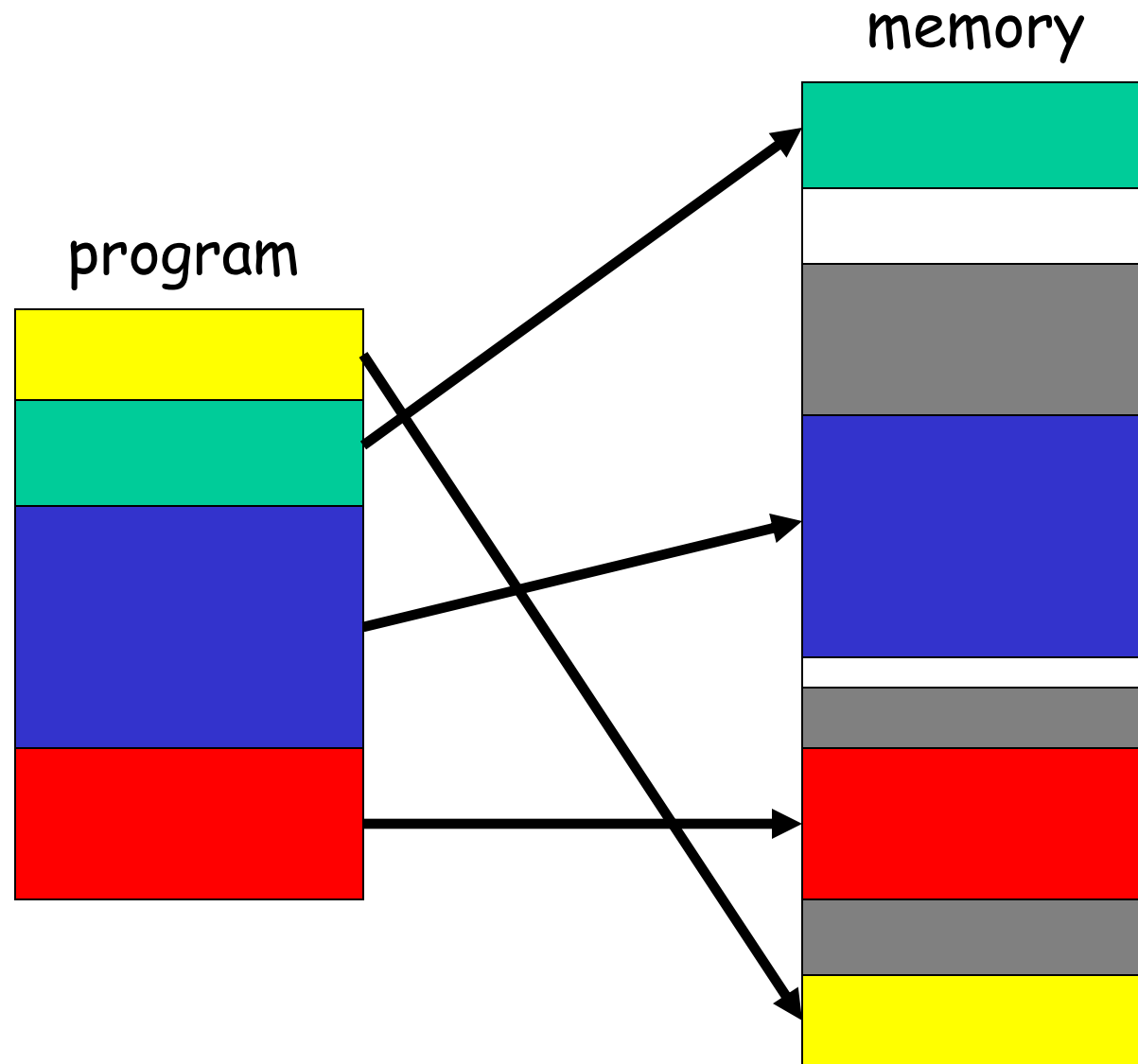
Memory Protection

- ❑ Tagging — specify protection of each address
 - + Extremely fine-grained protection
 - High overhead — can be reduced by tagging sections instead of individual addresses
 - Compatibility
- ❑ More common is segmentation and/or paging
 - Protection is not as flexible
 - But much more efficient

Segmentation

- ❑ Divide memory into logical units, such as
 - Single procedure
 - Data in one array, etc.
- ❑ Can enforce different access restrictions on different segments
- ❑ Any segment can be placed in any memory location (if location is large enough)
- ❑ OS keeps track of actual locations

Segmentation



Segmentation

- ❑ OS can place segments anywhere
- ❑ OS keeps track of segment locations as $\langle \text{segment}, \text{offset} \rangle$
- ❑ Segments can be moved in memory
- ❑ Segments can move out of memory
- ❑ All address references go thru OS

Segmentation Advantages

- ❑ Every address reference can be checked
 - Possible to achieve **complete mediation**
- ❑ Different protection can be applied to different segments
- ❑ Users can share access to segments
- ❑ Specific users can be restricted to specific segments

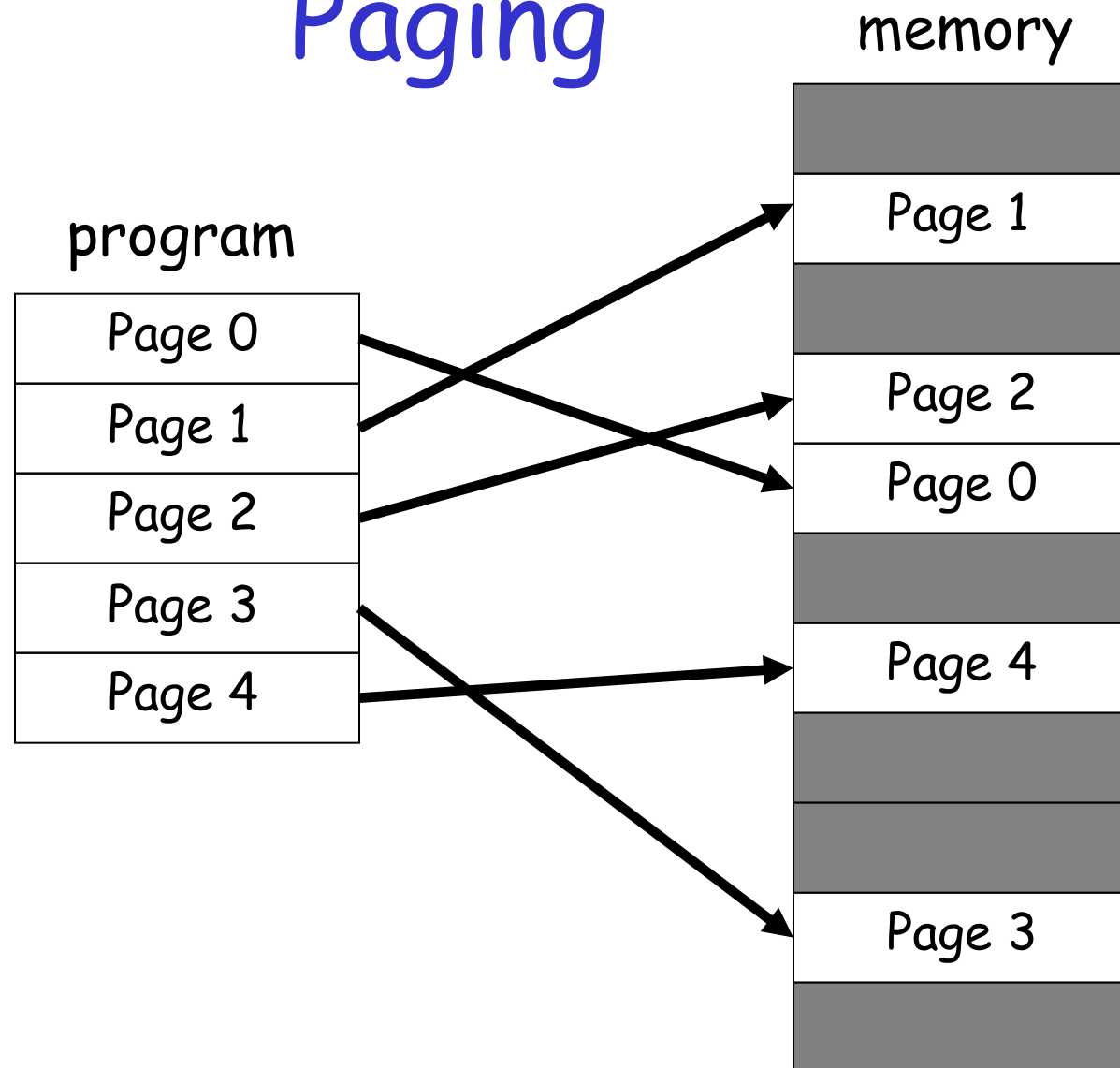
Segmentation Disadvantages

- ❑ How to reference $\langle \text{segment}, \text{offset} \rangle$?
 - OS must know segment **size** to verify access is within segment
 - But some segments can grow during execution (for example, dynamic memory allocation)
 - OS must keep track of **variable** segment sizes
- ❑ Memory fragmentation is also a problem
 - Compacting memory changes tables
- ❑ A lot of work for the OS
- ❑ More complex $\Rightarrow$ more chance for mistakes

Paging

- ❑ Like segmentation, but fixed-size segments
- ❑ Access via <page,offset>
- ❑ Plusses and minuses
 - + Avoids fragmentation, improved efficiency
 - + OS need not keep track of variable segment sizes
 - No logical unity to pages

Paging



Other OS Security Functions

- ❑ OS must enforce access control
- ❑ Authentication
 - Passwords, biometrics
 - Single sign-on, etc.
- ❑ Authorization
 - ACL
 - Capabilities
- ❑ OS is an attractive target for attack!